

---

# Byzantine Reliable Broadcast

Mohsen Lesani

# Byzantine reliable broadcast

---

- Almost the same primitive as the fail-silent model. It needs to reach agreement on delivered messages.
- To simplify the protocol, a Byzantine reliable broadcast instance is used to deliver **one message**.
- A priori declares a **sender process** for the instance

# Authenticated communication primitives

---

- Recall modules in model with crash failures
  - Perfect Links (pl)
  - Best-effort Broadcast (beb)
- Authenticated versions can be defined that tolerate attacks
  - Authenticated Perfect Links (al)
  - Authenticated Best-effort Broadcast (abeb)
- Implemented using cryptographic authentication (MACs or digital signatures)

# Byzantine broadcast variants

---

- Byzantine consistent broadcast
- Byzantine reliable broadcast

# Byzantine Consistent Broadcast (bcb)

---

## Events

- Request  $\langle \text{broadcast } (m) \rangle$   
Broadcasts a message  $m$  to all processes
- Indication  $\langle \text{deliver } (p, m) \rangle$   
Delivers a message  $m$  from sender  $p$

# Byzantine Consistent Broadcast (bcb)

---

## Properties:

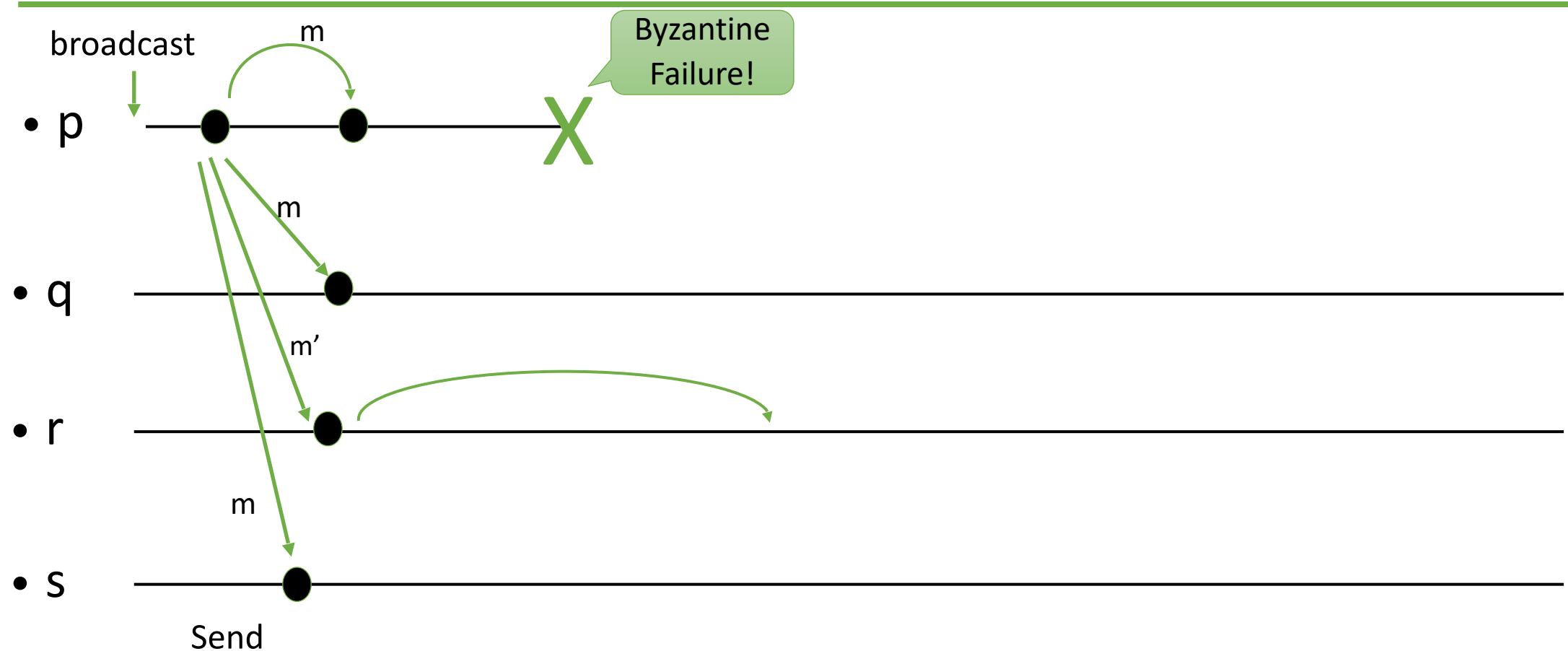
- BCB1 (Validity) = BEB1:  
Every message broadcast by a correct process is eventually delivered by every correct process.
- BCB2 (No duplication):  
Every correct process delivers at most one message.
- BCB3 (Integrity):  
If a correct process delivers  $m$  with sender  $p$ , and  $p$  is correct, then  $p$  has broadcast  $m$ .
- BCB4 (**Consistency**):  
If a correct process delivers message  $m$  and another correct process delivers message  $m'$ , then  $m=m'$ .
- Note: Some correct process may not deliver any message (agreement is not required yet)

# Using Byzantine Quorums

---

- System of  $N > 3f$  processes,  $f$  are faulty.
- Let's have  $N = 3f + 1$  for simplicity.
- Every subset with size larger than or equal to  $2f+1$  processes is a quorum.
- The collection of all quorums is a quorum system.

# Example

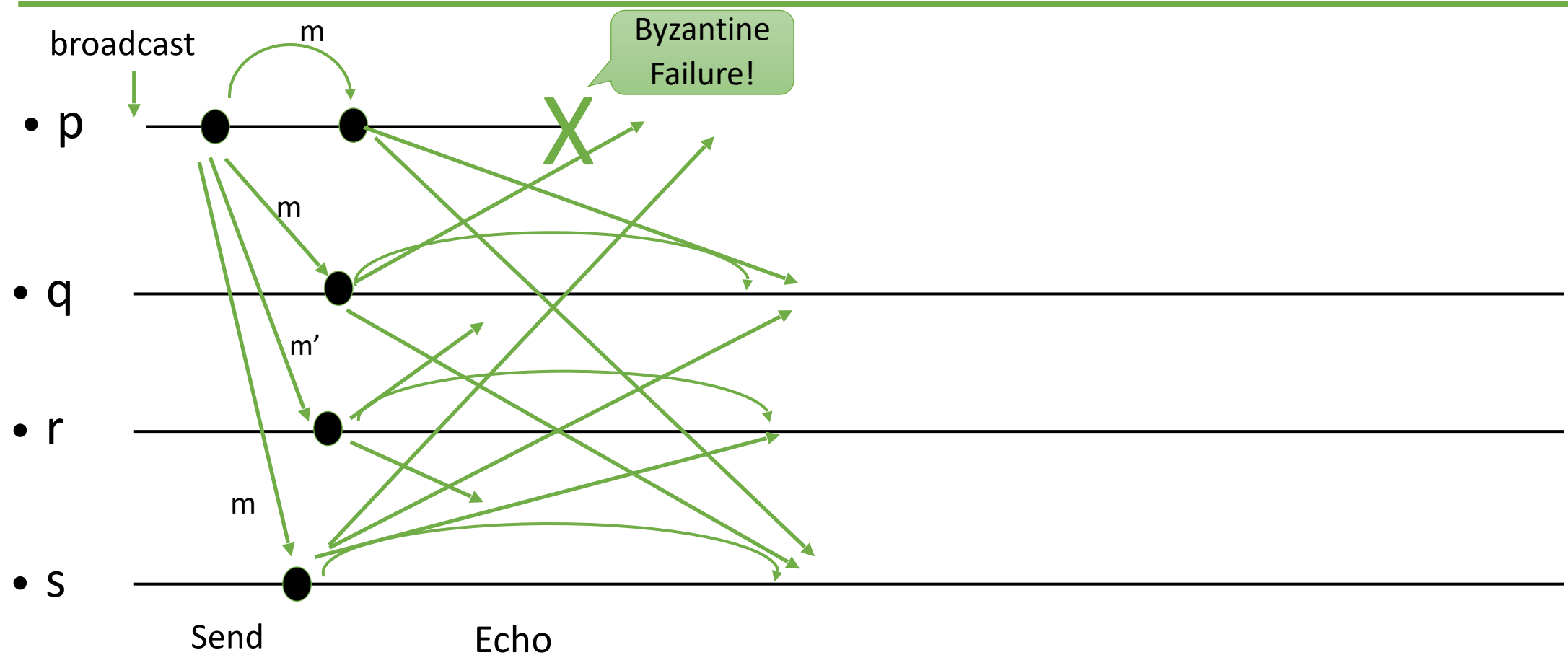


- Faulty sender p sends two different messages **m** and **m'**.
- Processes q and s deliver the message **m**.

- Process r does not deliver any message. It does not receive a quorum.
- 2 rounds,  $O(n^2)$  messages  
 $O(n^2 |m|)$  communicated data



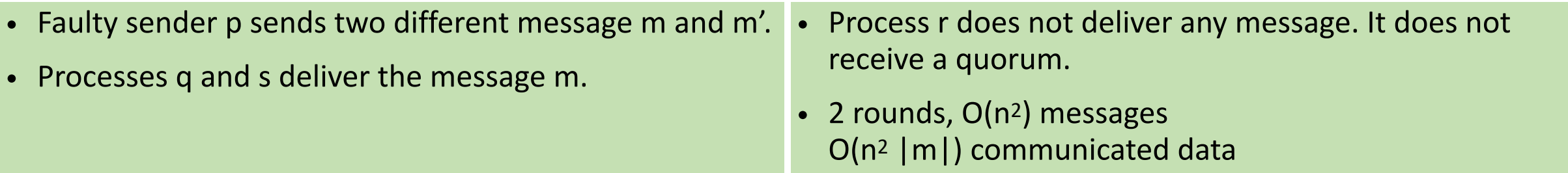
# Example



- Faulty sender p sends two different messages m and m'.
- Processes q and s deliver the message m.

- Process r does not deliver any message. It does not receive a quorum.
- 2 rounds,  $O(n^2)$  messages  
 $O(n^2 |m|)$  communicated data

Byzantine Failure!



- Process  $r$  does not deliver any message. It does not receive a quorum.
- 2 rounds,  $O(n^2)$  messages  
 $O(n^2 \cdot |m|)$  communicated data

# Authenticated Echo Broadcast

---

**Implements** bcb, **uses** abeb, with sender  $s$  (where  $N = 3f + 1$ )

**upon** <bcb, broadcast ( $m$ )> **do**  
    **trigger** <abeb, broadcast(Send( $m$ ))>

**upon** <abeb, deliver( $s$ , Send( $m$ ))> **do**  
    **trigger** <abeb, broadcast(Echo( $m$ ))>

**upon** <abeb, deliver( $p$ , Echo( $m$ ))> **do**  
    echo[ $p$ ] :=  $m$   
    **if**  $\exists m : \#\{p \mid \text{echo}[p]=m\} \geq 2f + 1$  **then**  
        **trigger** <deliver( $s$ ,  $m$ )>

Note: The code to prevent duplicate execution is omitted.

# Using Byzantine Quorums

---

- Two distinct quorums together contain more than  $4f+2$  processes.
- Thus, they overlap in at least  $f + 1$  processes.
- At least one of them is a correct process.

Proof of the consistency property:

- The correct process in the intersection has broadcast the same message  $Echo[m]$  to all processes.
- Therefore, the two processes deliver the same message  $m$ .

# Byzantine Reliable Broadcast (brb)

---

## Events

- Request  $\langle \text{broadcast}(m) \rangle$
- Indication  $\langle \text{deliver}(p, m) \rangle$

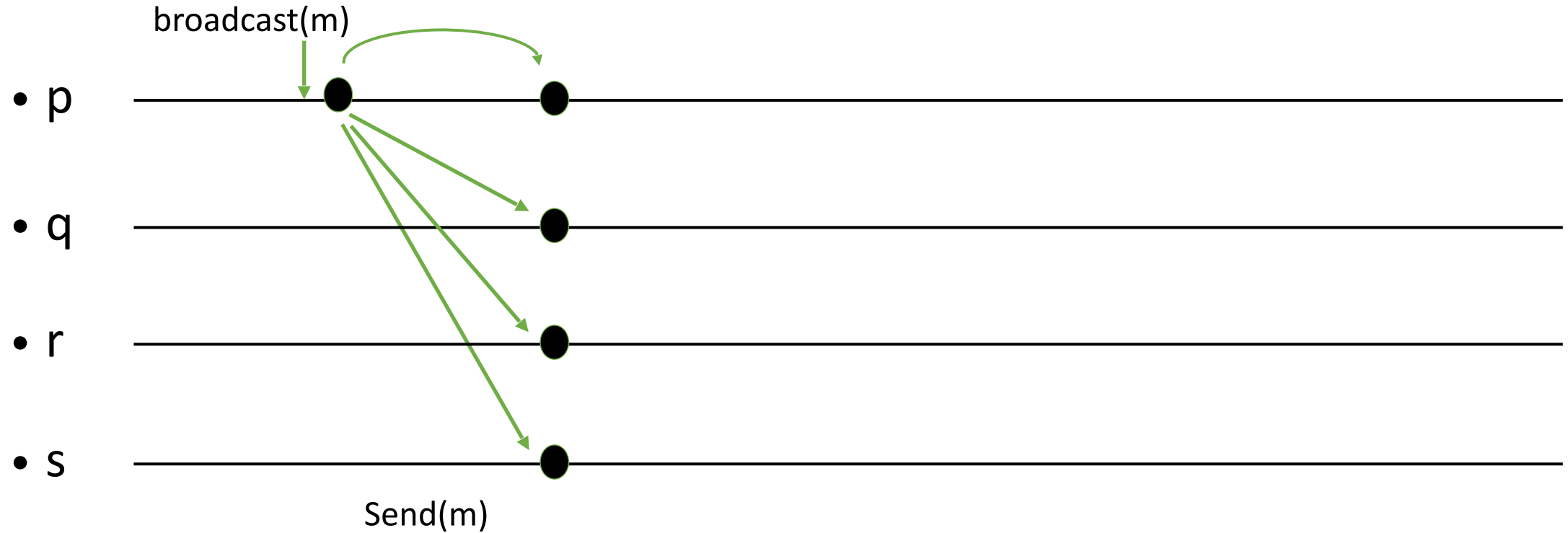
## Properties

- BRB1 (Validity) = BCB1
- BRB2 (No duplication) = BCB2
- BRB3 (Integrity) = BCB3
- BRB4 (Consistency) = BCB4
- BRB5 (**Totality**):

If some correct process delivers a message, then every correct process eventually delivers a message.

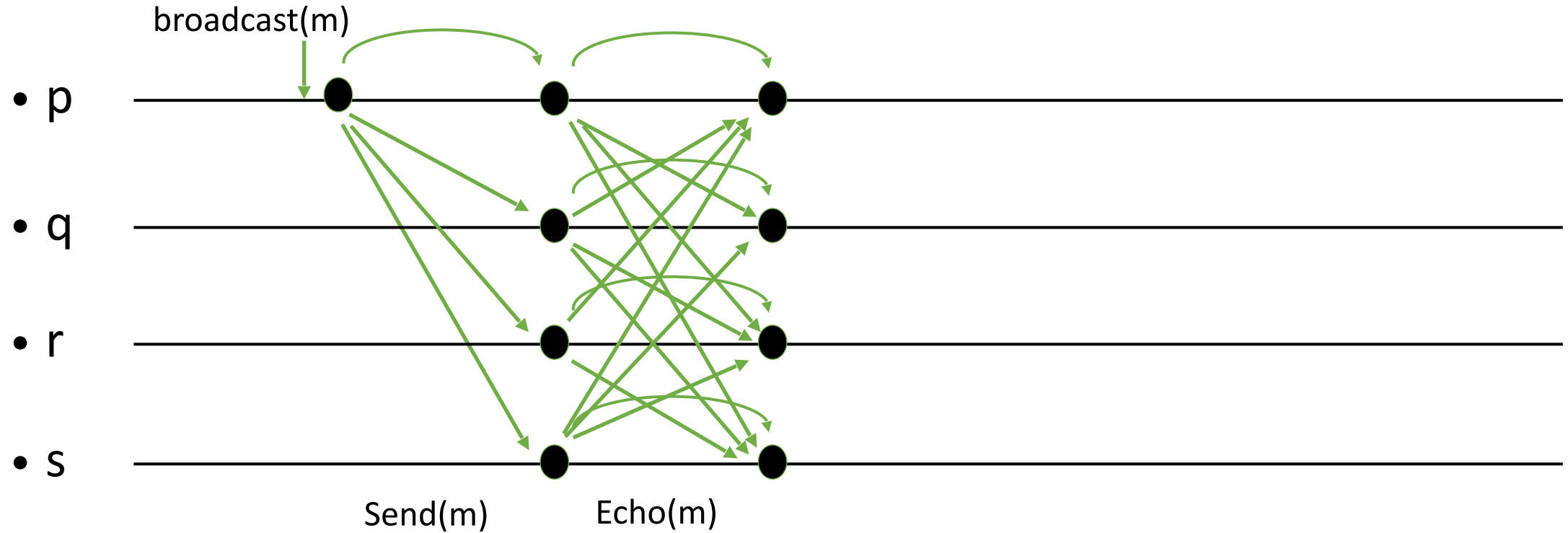
By totality, either all or none of the correct processes deliver a message. By consistency, they deliver the same message.

# Example



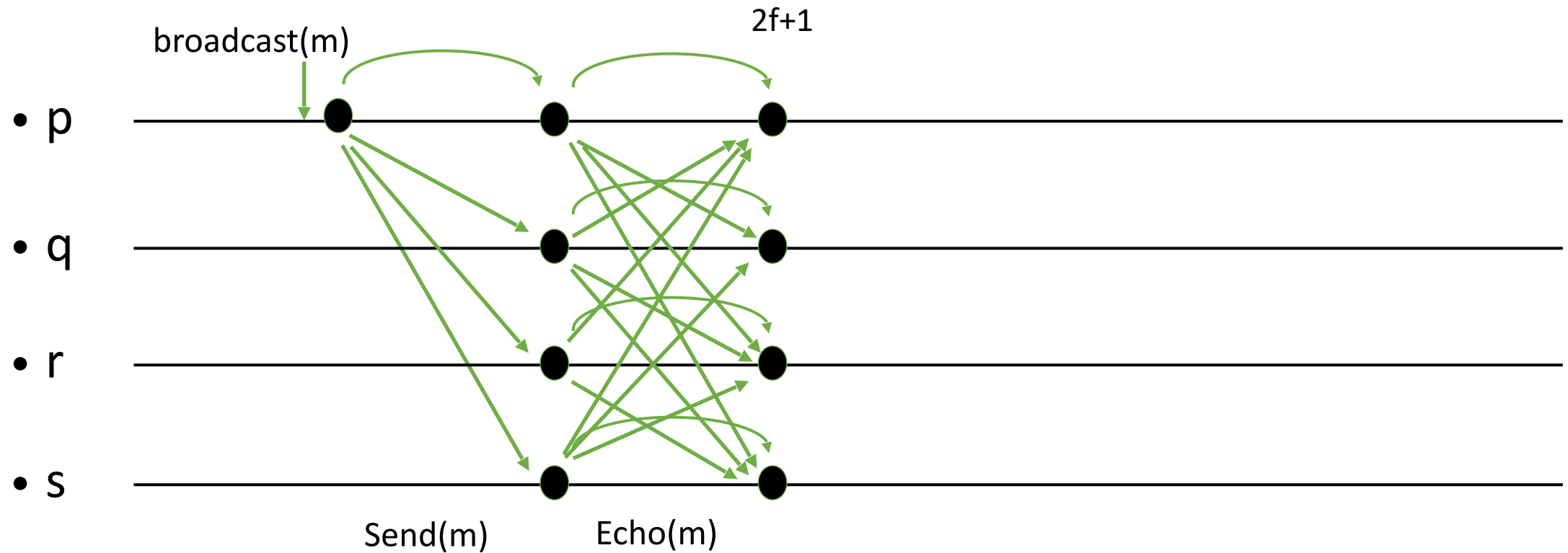
3 rounds;  $O(n^2)$  messages;  $O(n^2 |m|)$  communication

# Example



3 rounds;  $O(n^2)$  messages;  $O(n^2 |m|)$  communication

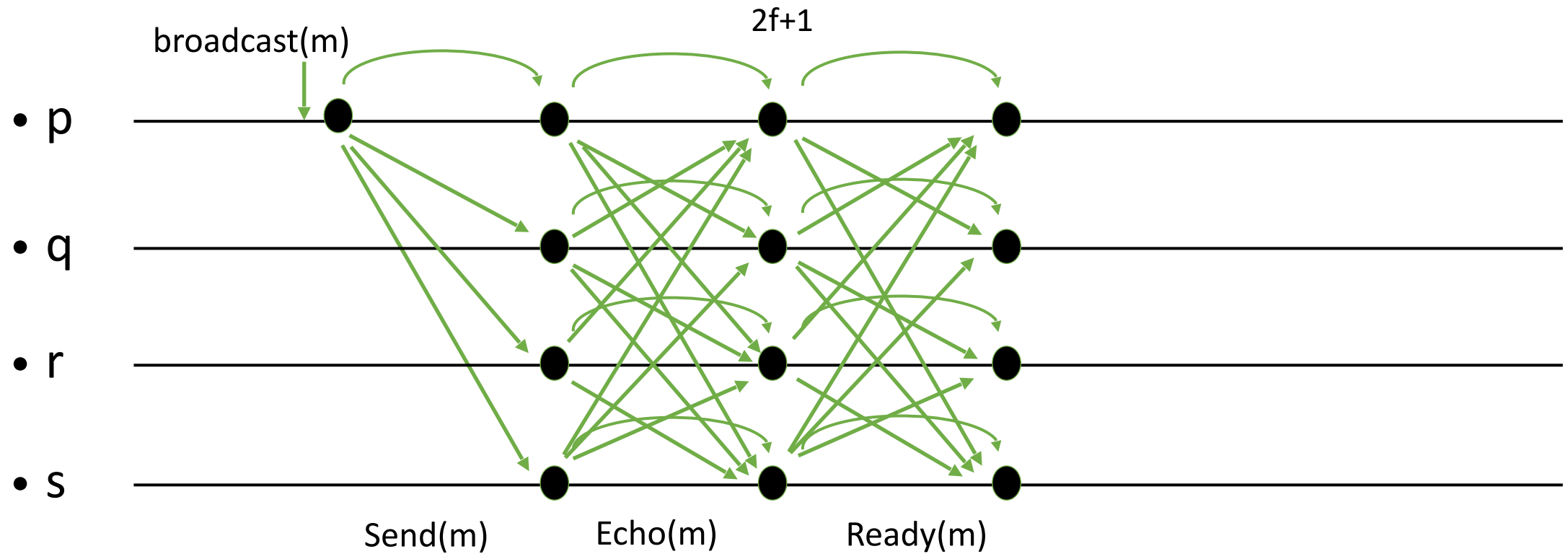
# Example



3 rounds;  $O(n^2)$  messages;  $O(n^2 |m|)$  communication

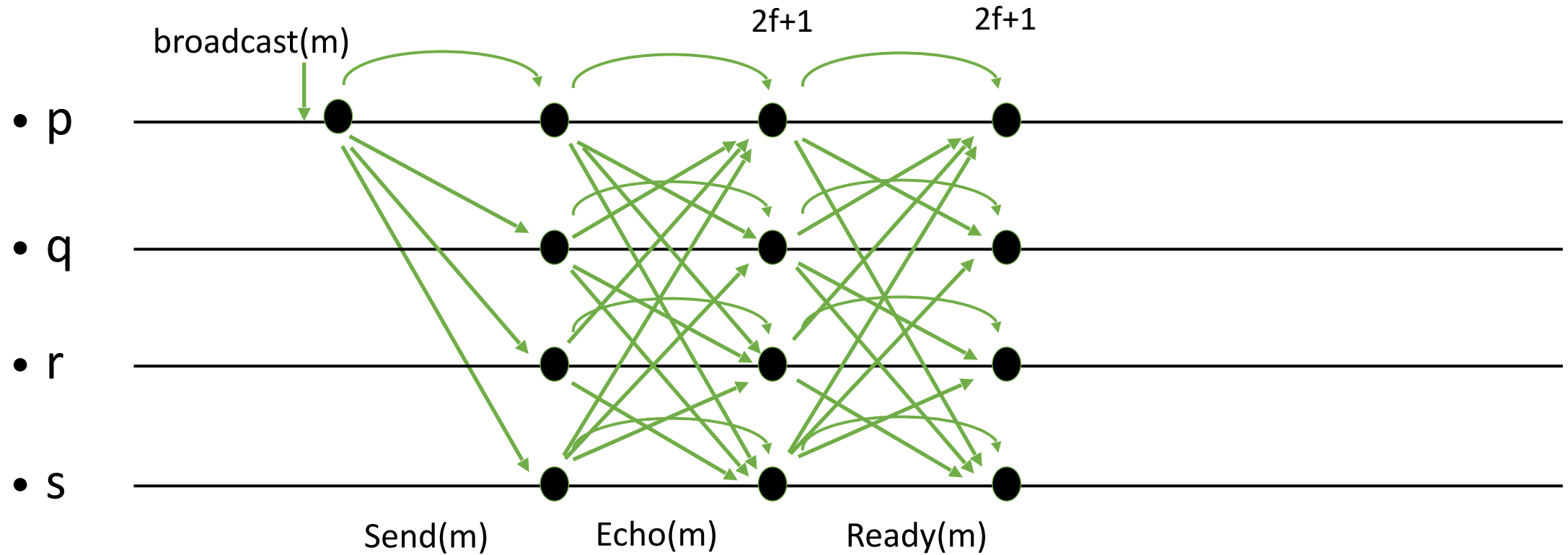


# Example



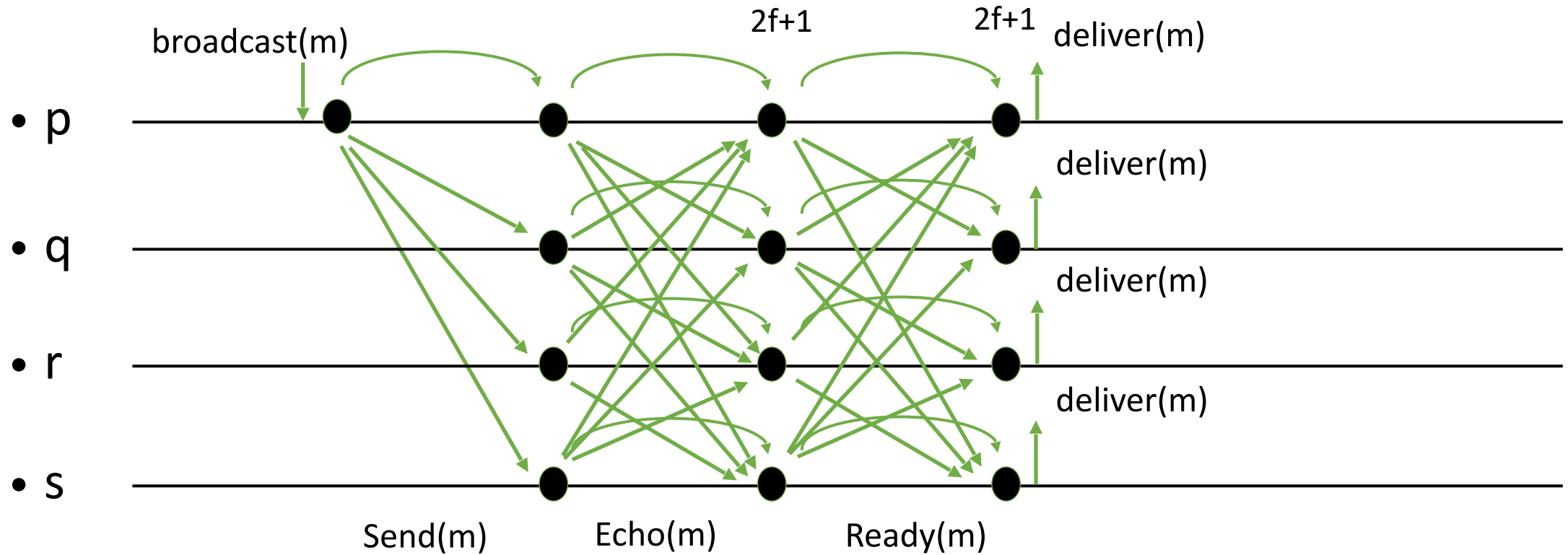
3 rounds;  $O(n^2)$  messages;  $O(n^2 |m|)$  communication

# Example



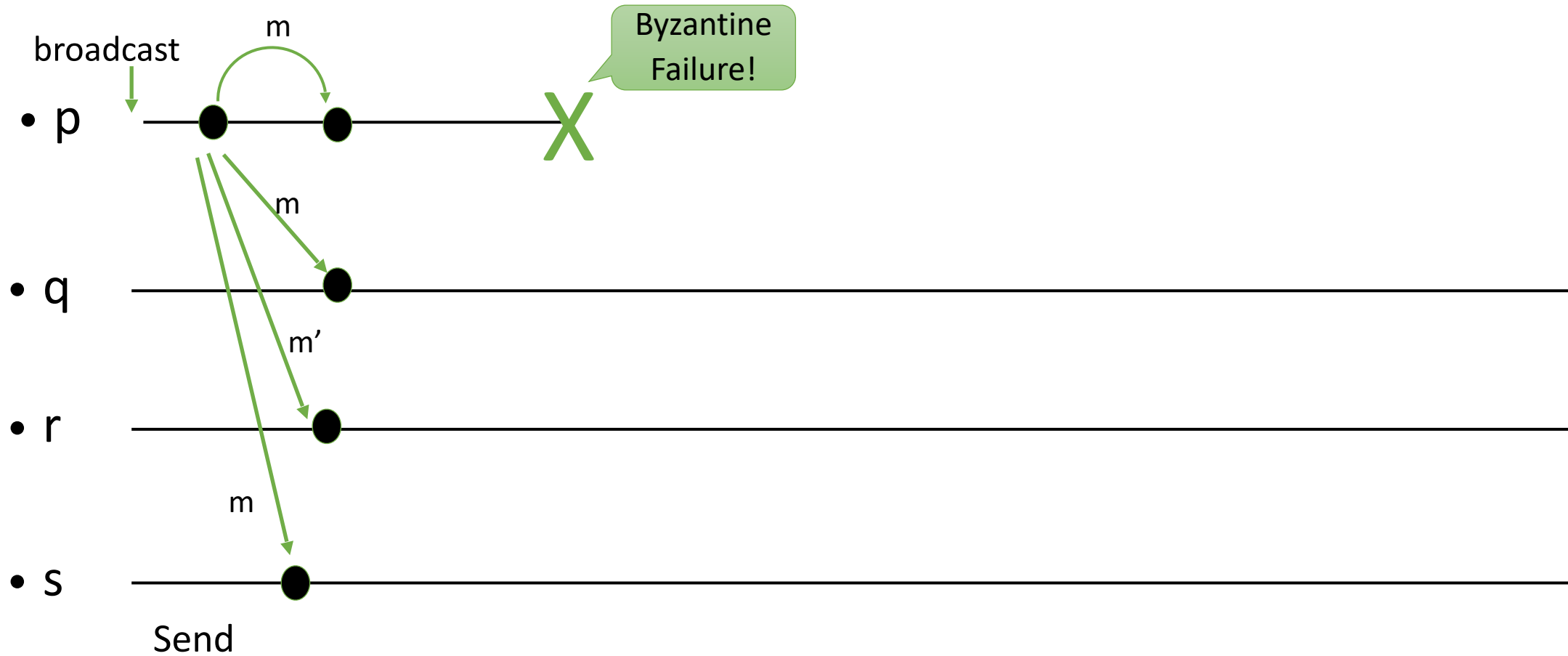
3 rounds;  $O(n^2)$  messages;  $O(n^2 |m|)$  communication

# Example

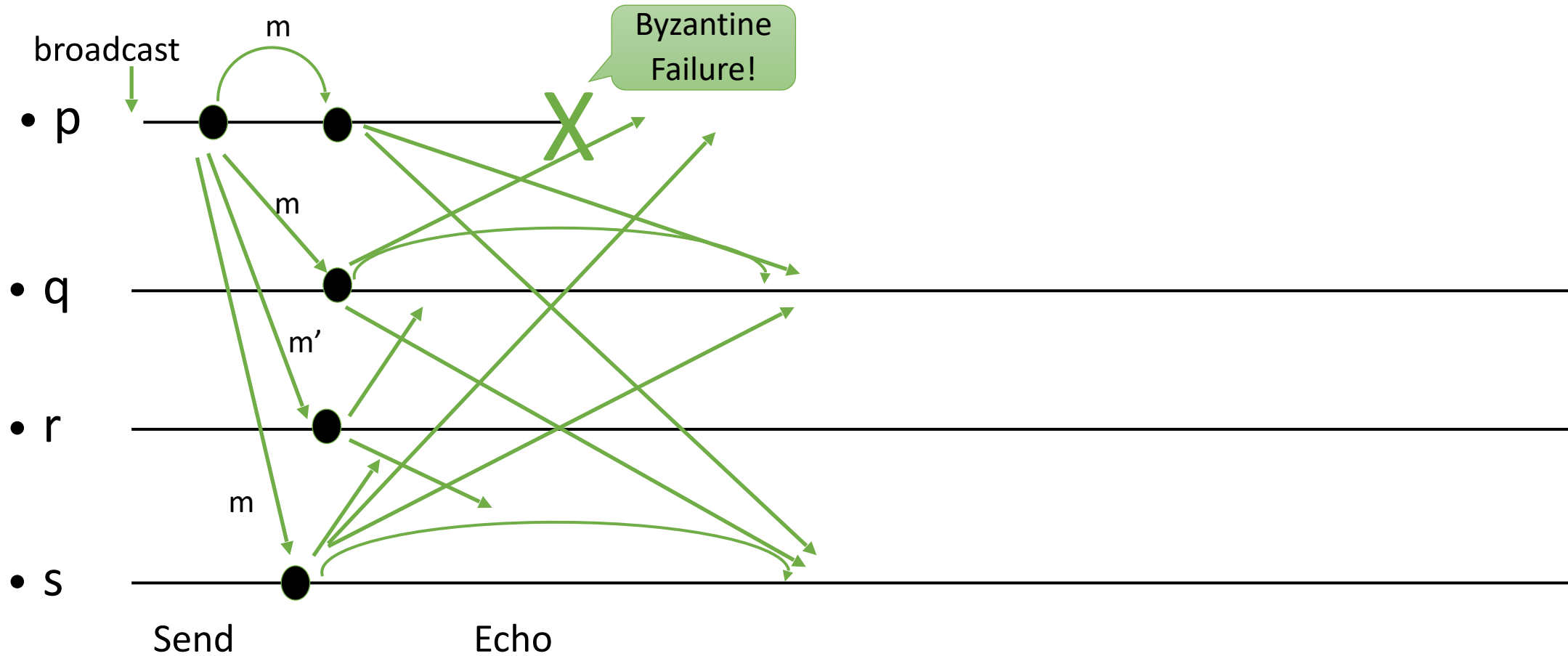


3 rounds;  $O(n^2)$  messages;  $O(n^2 |m|)$  communication

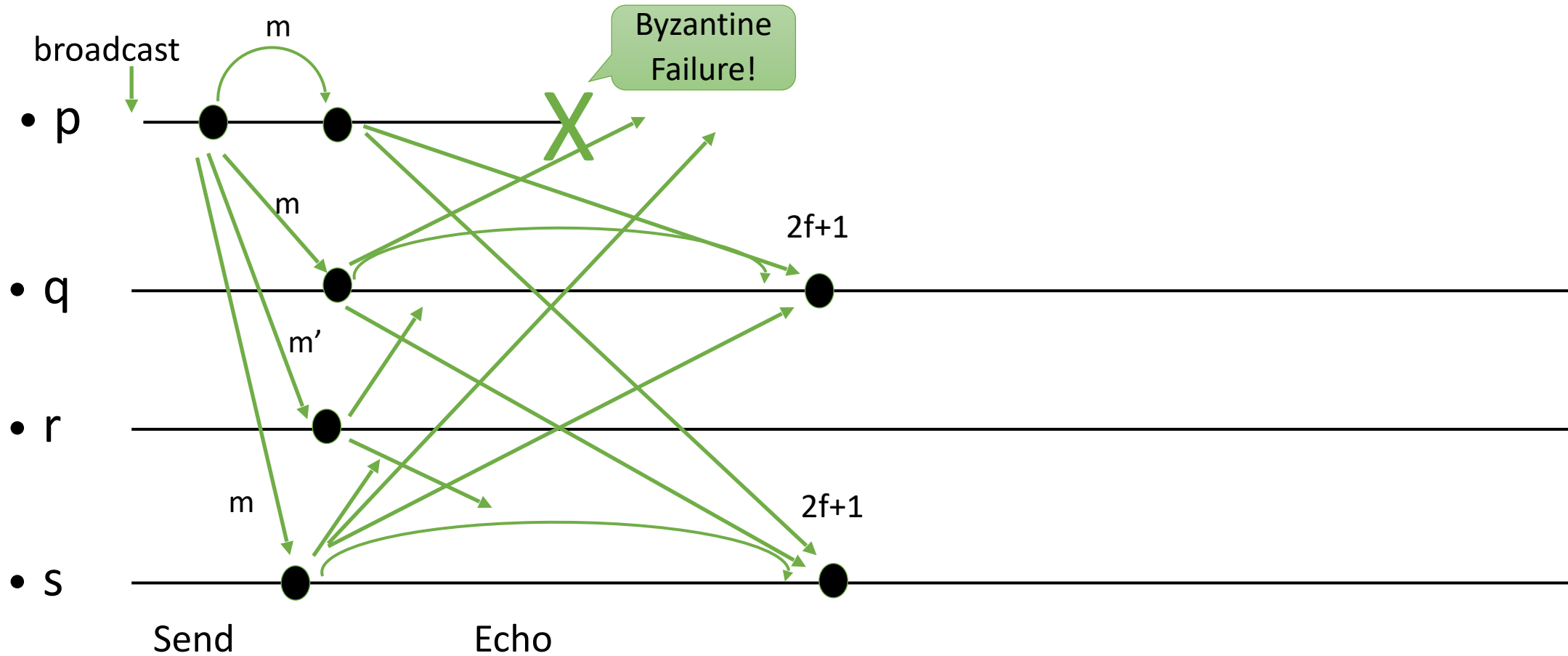
# Example



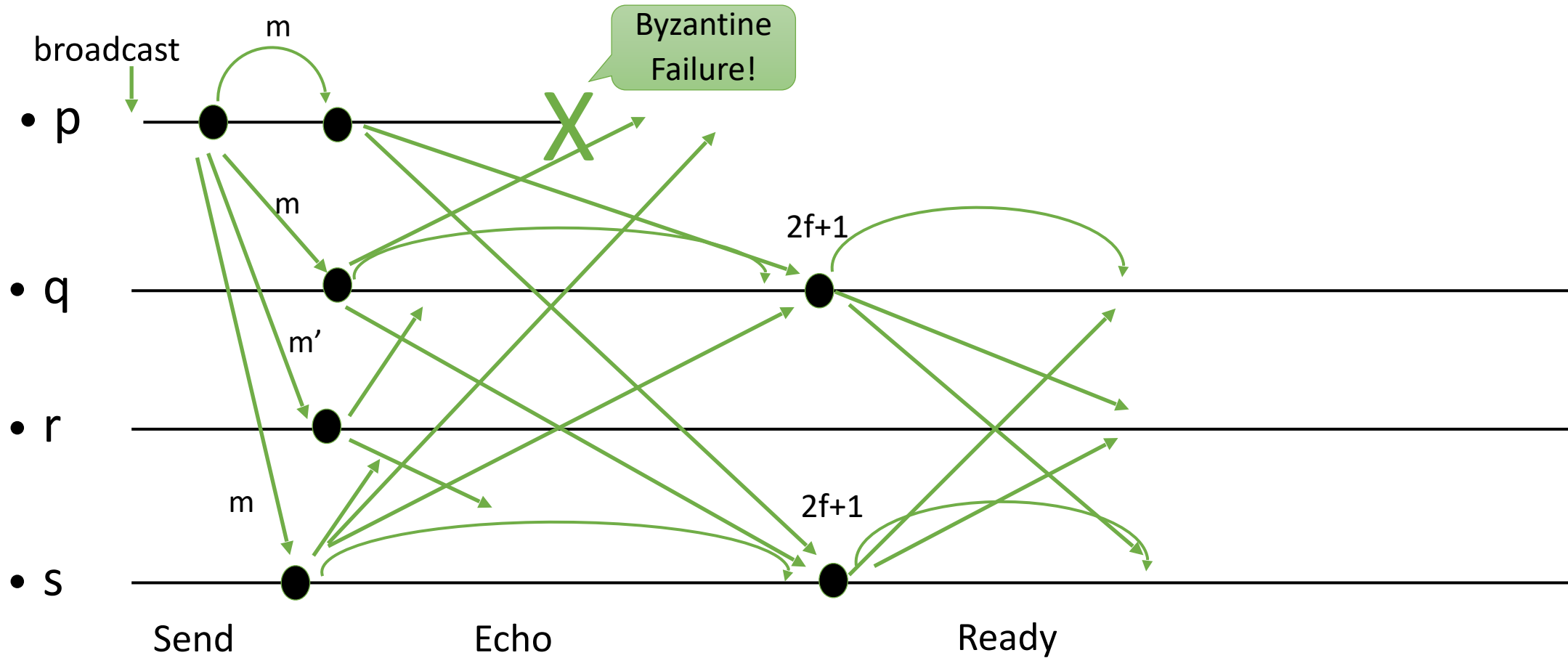
# Example

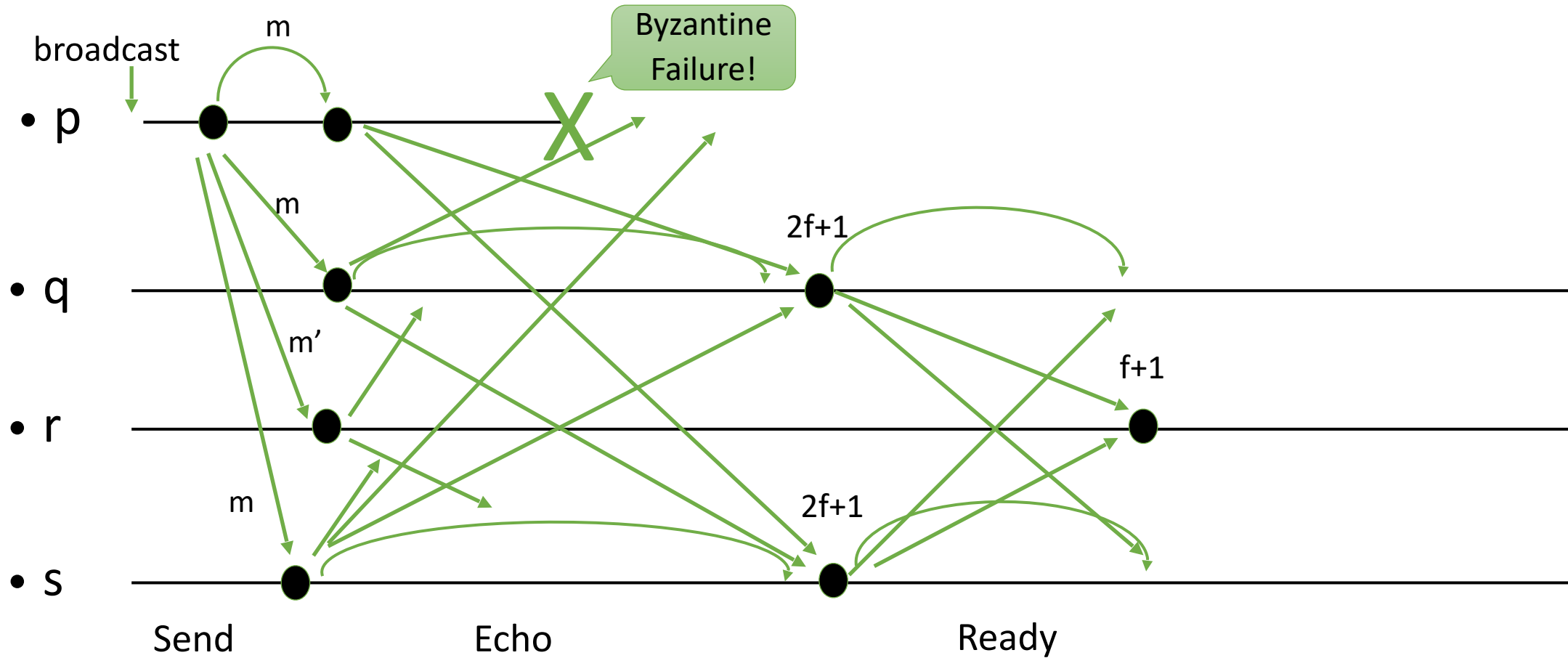


# Example



# Example

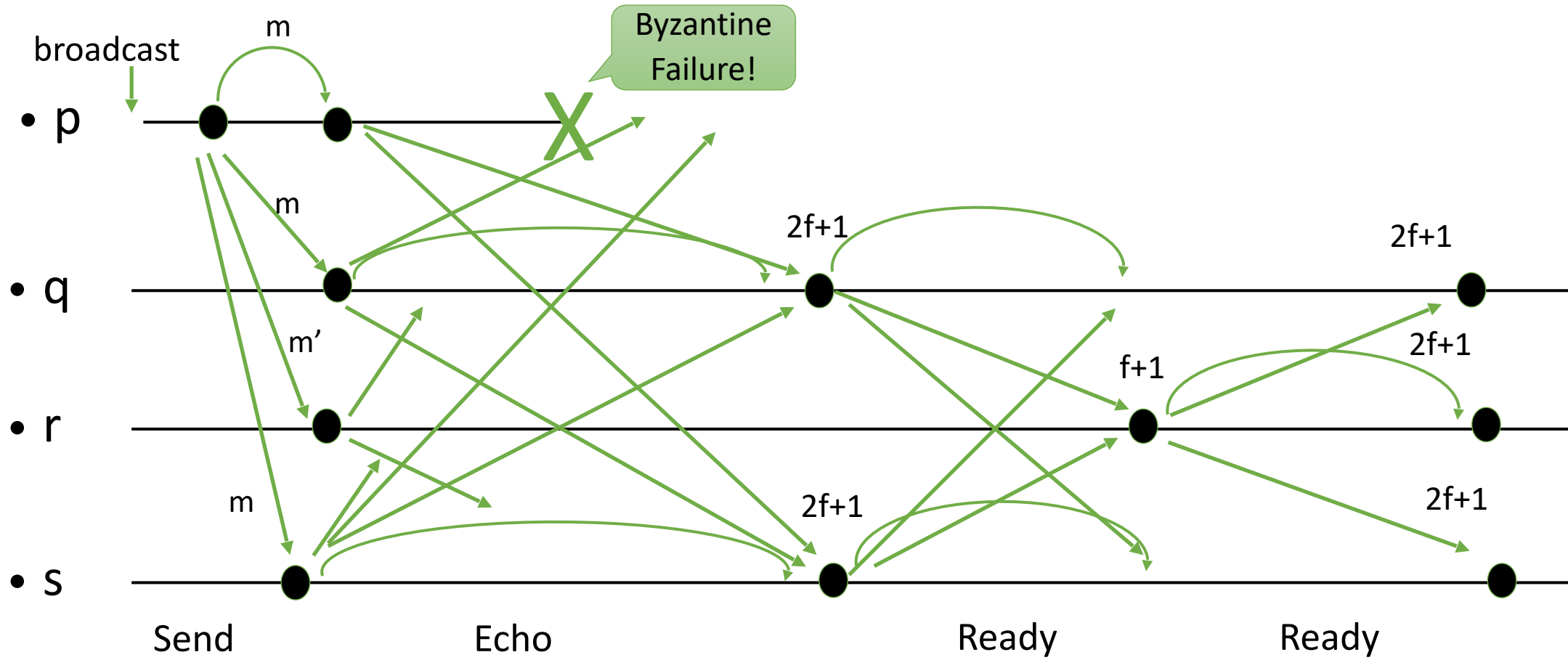


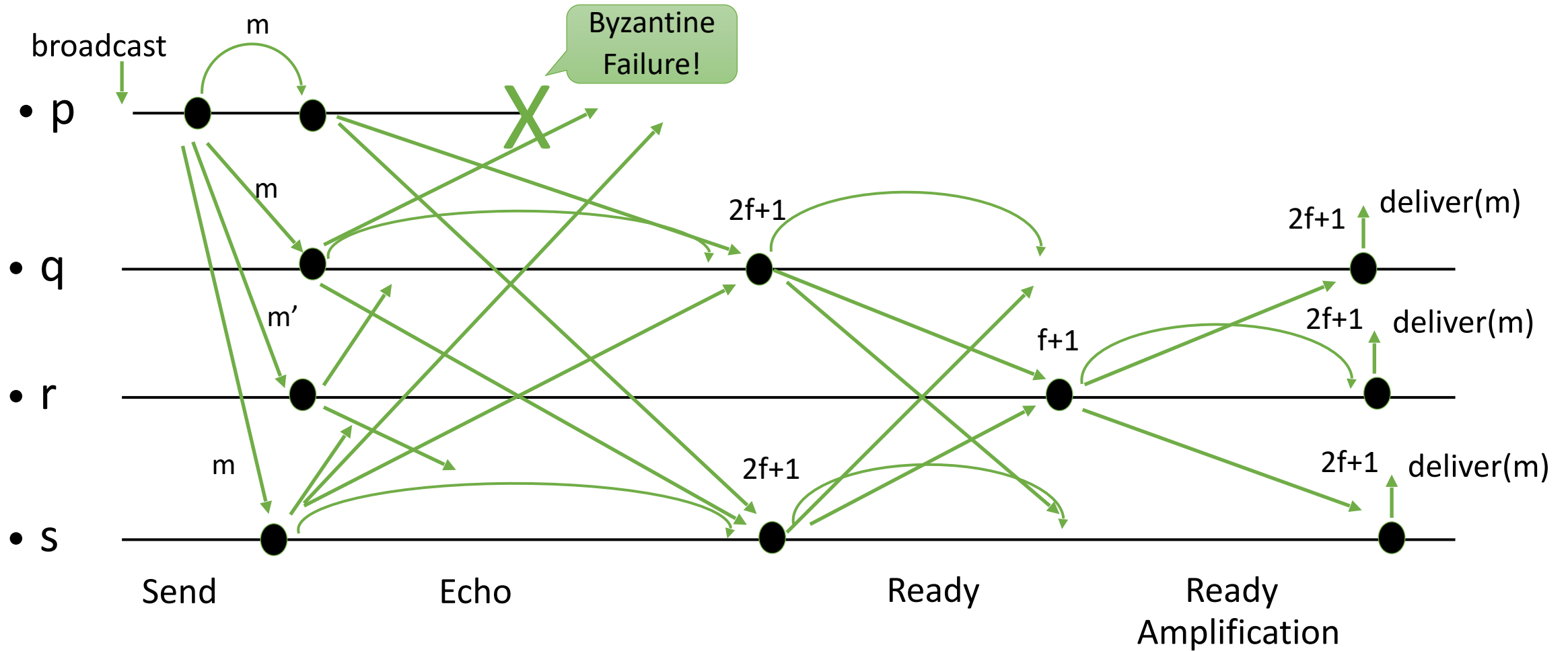






# Example





# Authenticated Double-Echo Broadcast (Bracha)

---

**Implements** brb, **uses** abeb, with sender  $s$  ( $N > 3f$ )

sent-ready := **false**

upon <brb, broadcast( $m$ )> do  
    **trigger** <abeb, broadcast(Send[ $m$ ])>

**upon** <abeb, deliver( $s$ , Send( $m$ ))> **do**  
    **trigger** <abeb, broadcast(Echo[ $m$ ])>

**upon** <abeb, deliver( $p$ , Echo( $m$ ))> **do**  
    echo[ $p$ ] :=  $m$   
    **if**  $\exists m : \#\{p \mid \text{echo}[p]=m\} > 2f \wedge \neg \text{sent-ready}$  **then**  
        sent-ready := **true**  
        **trigger** <abeb, broadcast(Ready( $m$ ))>

# Authenticated Double-Echo Broadcast (Bracha)

---

```
upon <abeb, deliver(p, Ready(m))> do  
  ready[p] := m  
  if  $\exists m : \#\{p \mid \text{ready}[p]=m\} > f \wedge \neg \text{sent-ready}$  then  
    ▷ amplification of READY messages  
    sent-ready := true  
    trigger <abeb, broadcast(Ready(m))>  
  else if  $\exists m : \#\{p \mid \text{ready}[p]=m\} > 2f$  then  
    trigger <deliver(s, m)>
```

Note: some code to prevent duplicate execution is omitted.

# Amplification

---

## Totality by Amplification:

- A correct process has delivered the message. In the  $2f+1$  processes that it received Ready from, there are at least  $f+1$  correct processes. These correct processes send Ready to all processes.
- After receiving  $f+1$  Ready messages, all correct processes amplify the Ready message if they have not already sent it.
- There are at least  $2f+1$  correct processes, and they all send the Ready message.
- Therefore, each correct process receives Ready from  $2f+1$  processes and delivers the message.

# Byzantine Broadcast Channel

---

- Implemented by a sequence of one-message instances of Byzantine broadcasts for each process
- Every message is delivered together with a unique label
  - Consistency and totality hold for each label
- Two variants
  - Consistent Channel
  - Reliable Channel

# Interface and properties of Byzantine consistent channel

---

## Module:

**Name:** ByzantineConsistentBroadcastChannel, instance bcch.

We introduce labels to uniquely identify deliveries.

## Events:

- **Request:**  $\langle \text{bcch}, \text{broadcast}(m) \rangle$ : Broadcasts a message  $m$  to all processes.
- **Indication:**  $\langle \text{bcch}, \text{deliver}(p, \ell, m) \rangle$ : Delivers a message  $m$  with label  $\ell$  broadcast by process  $p$ .

## Properties:

- **BCCH1:** Validity: If a correct process  $p$  broadcasts a message  $m$ , then every correct process eventually delivers  $m$ .
- **BCCH2:** No duplication: For every process  $p$  and label  $\ell$ , every correct process delivers at most *one message with label  $\ell$*  and sender  $p$ .
- **BCCH3:** Integrity: If some correct process delivers a message  $m$  with sender  $p$ , and process  $p$  is correct, then  $m$  was previously broadcast by  $p$ .
- **BCCH4:** Consistency: If some correct process delivers a message  *$m$  with label  $\ell$*  and sender  $s$ , and another correct process delivers a message  *$m'$  with label  $\ell$*  and sender  $s$ , then  $m = m'$ .



# Byzantine Consistent Channel

---

## Implements:

ByzantineConsistentBroadcastChannel, instance bcch.

## Uses:

ByzantineConsistentBroadcast (multiple instances).

**upon event**  $\langle \text{init} \rangle$  **do**

$n := [0]^N$

**ready** := **true**

**forall**  $p \in \Pi$  **do**

Initialize a new instance  $\text{bcb}[p, 0]$  of

ByzantineConsistentBroadcast with sender  $p$

$n$ : It keeps the number of the current broadcast instance for each process.  
 $\text{ready}$ : It is used to wait for the previous broadcast instance to finish.

# Byzantine Consistent Channel

---

**upon event**  $\langle \text{broadcast}(m) \rangle$  **such that**  $\text{ready} = \text{true}$  **do**

**trigger**  $\langle \text{bcb}[\text{self}, n[\text{self}]], \text{broadcast}(m) \rangle$

$\text{ready} := \text{false}$

**upon event**  $\langle \text{bcb}[p, n[p]], \text{deliver}(p, m) \rangle$  **do**

**trigger**  $\langle \text{deliver}(p, n[p], m) \rangle$

$n[p] := n[p] + 1$

    Initialize a new instance  $\text{bcb}[p, n[p]]$  of ByzantineConsistentBroadcast with sender  $p$

**if**  $p = \text{self}$  **then**

$\text{ready} := \text{true}$